

MAVRA

让数据智能体共享的指标定义 在数据变化后依然正确

Keeping Shared Metric Definitions Valid for Data Agents

2026 / 10 / 12 · 组会汇报 · 投稿目标 SIGMOD 2027

背景 → 问题 → 做法 → 实验 → 进展与讨论

一页看懂：问题、做法和效果

1 背景

大模型智能体靠探索数据库来回答业务问题。

“退货金额怎么算”这类口径是业务知识，表结构里没有；学一次很贵，所以越来越多系统让智能体把学到的口径共享给后来的会话。

2 问题

共享的口径是在某一时刻的数据上学到的。

日常的数据更新（比如多了一种状态行）不改表结构，却让共享的 SQL 照常执行、悄悄返回错误汇总值；用它的智能体看不出来。

3 做法

让数据库替智能体记住“定义为什么成立”：

把成立条件记录为可执行检查，随数据版本维护，查询只在检查通过的数据上执行；坏了只做唯一且过回归的修复，否则撤下。

4 效果

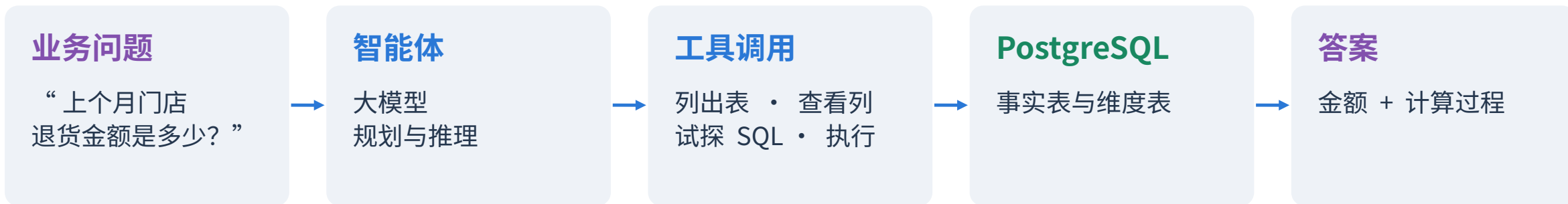
破坏性数据变化下，智能体正确率 73%，对照的轨迹检索 57%。

并发写入下零次在违规数据上作答。

使用者的任务从 64.47 秒缩到 37.68 秒，token 少 40.7%。

一句话：共享让智能体少探索；维护让共享的知识在变化的数据上继续成立。

数据智能体：先摸清数据库，再写 SQL 回答业务问题



每道题都要摸清的事

- ① 哪张表装着退货，哪一列是金额
- ② 一笔退货对应几行（粒度）
- ③ 用哪个日期把退货归到“上个月”
- ④ 哪些行不该算（取消、申请中……）

5.6 轮

不共享知识时
每道题的工具交互轮数

32%

不共享知识时
端到端正确率

真正难的不是写 SQL，而是弄清“口径”：哪些行算一次、怎么归期、怎么过滤。

“口径”是业务知识，表结构里没有

退货金额的口径

事实表 store_returns
度量 SUM(sr_return_amt)
粒度 小票号 + 商品 = 一笔退货
时间 按退货日期归期
过滤 只算“已完成”的退货

这五条 SQL 里不会写明，表结构也不强制。

21.3%

Spider 2.0 企业级任务上
o1-preview 智能体的完成率

34.9% → 54.9%

BIRD：给 GPT-4 专家标注的
业务知识前后的正确率

32% → 75%

我们的实验：新会话拿到
共享口径前后的正确率

口径必须有人告诉智能体，或者由它自己摸索出来再记住，否则每道题都从零开始。

趋势：把智能体学到的口径共享给后来的会话

A 人工声明的指标层

Databricks metric views、dbt 语义层：
工程师手工写出度量、键和关联。

32% → 75%

共享口径后的端到端正确率

B 从历史里推导

DataLab：用大模型从脚本历史和血缘
生成派生列的计算逻辑。

5.6 → 3.7 轮

每道留出题的工具交互轮数

C 复用智能体轨迹

AgentSM、智能体记忆系统：
把成功轨迹存起来，按题面检索复用。

-36%

每道留出题的输入 token

共享的口径成了“长期状态”，可它底下的数据库每天都在变。

SQL 照常执行，退货金额却被算了两次

1 Agent A 学到口径

每笔退货 = 一条“完成”记录
小票号 + 商品唯一
SQL: SUM(sr_return_amt)

2 ETL 追加状态流水

每笔新退货多一条“申请中”行
键不再唯一
表名、列名、类型都没变

3 Agent B 复用 SQL

学习者的会话早已结束
新会话没有基准真值
执行成功，看不出重复计数

6,588,699.86

共享 SQL 返回的单期退货金额

3,294,349.93

该期间真实的已完成退货金额

共享保存了业务知识，也让一次过期的观测传播给之后的每个智能体。

为什么现有手段挡不住

1 按表结构跟踪

表名、列名、类型都没变，
所以什么也看不到。

645 个错误答案

回放实验，只看结构

2 数据质量测试

dbt 式按表测试：不知道
哪个口径依赖哪条性质，
失败只能整表隔离。

少答对 342 题

都是能修复或本可保留的

3 指标层靠人声明

连接性质靠人声明。Databricks 文档：
该性质“不在运行时验证”。

声明 $\neq$ 核对

放大行数时度量直接错

4 智能体自己检查

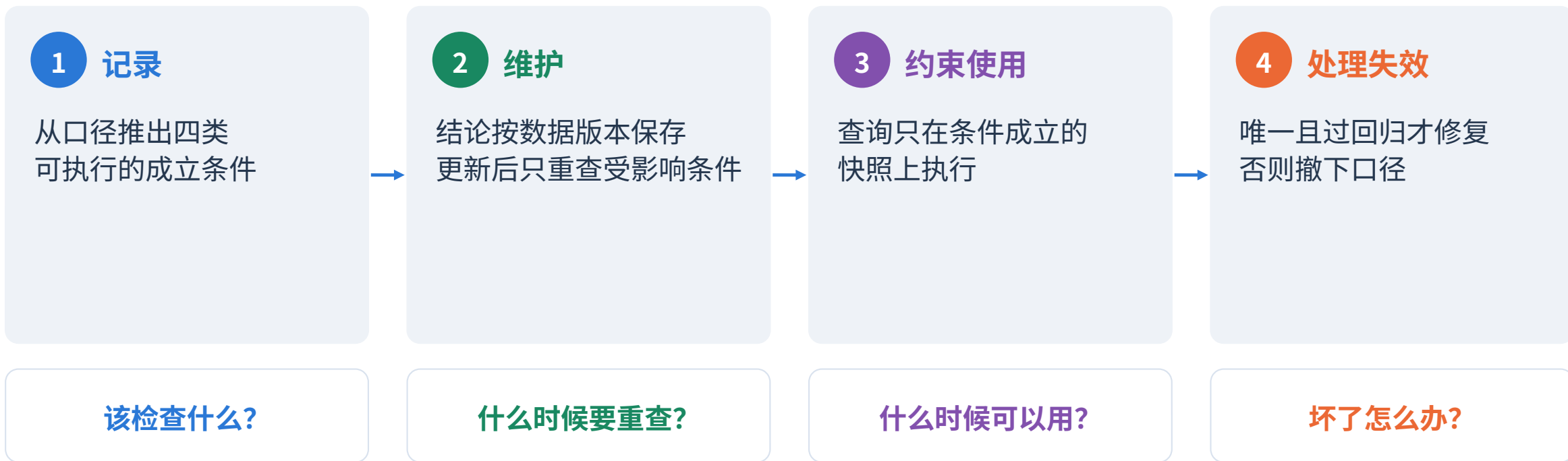
只有 16% 的会话查键唯一性；提示它检查，也要
每次重新找出区分列。

57% $\rightarrow$ 63%

系统维护时为 73%

缺的是一个专门记录、维护“口径成立条件”并在使用时核对的机制。

我们的做法：数据库替智能体记住“定义为什么成立”



口径的有效性证据成为数据库里一等的、带版本的状态，而不是留在某次探索的轨迹里。

对应论文三项贡献：条件与充分性命题、绑定快照的执行与复用引理、有界修复算法。

记录：四类条件，用退货金额来看

条件	白话	退货金额里是	怎么查
过滤后键唯一	一笔业务事件只有一行	“已完成”过滤后，小票号 + 商品唯一	按键分组找重复
连接多重性	关联维度表不会放大行数	退货 → 商品表是多对一	维表侧的键唯一
时间角色	事实按正确的日期归期	退货日期 → 日期维度多对一	日期键多对一
日期覆盖	事实不会悄悄掉出所有期间	匹配不到日期的比例不高于准入时	比例对比

命题 1（充分性）

在业务前提下，四类条件成立 ⇒ 规范 SQL 把每笔业务事件在其期间恰好计数一次，遗漏仅限覆盖条件约束的部分。

边界也由命题划定：单位换算、两份一样合法的总体、合法但错误的日期键，条件看不到。

维护：更新触及哪些条件，就只重查哪些

1 表的版本变了

写事务里递增的事务性版本，
或默认的 DML 统计

3 重查并保存结论

索引 = (条件身份, 依赖表版本)
同一检查并发到达时只跑一次

2 找到读这张表的条件

只有它们进入重查；
其余条件的结论保留

4 所有口径共享结论

一个结论服务所有需要
同一条件的口径和智能体

19 次 → 3 次

更新 store_returns：8 个口径受影响
逐口径重查 19 次，不同条件只有 3 个

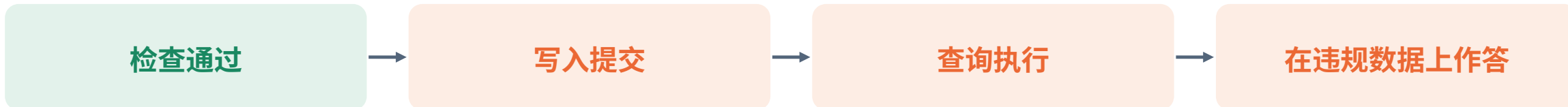
175 / 65

TPC-DS 的 99 个模板：口径数 / 不同条件数
平均每个条件被 8.6 个口径共享

维护工作量按“不同条件”计，不按口径数计；这部分节省通用的版本缓存也能拿到。

约束使用：检查和查询必须看同一份数据

执行前检查（常见做法）



绑定快照执行（MAVRA）



引理 1（结论复用）

两个快照里，条件读到的每张表的事务性版本都相同 ⇒ 条件在两个快照上的结论相同。

为什么要事务性版本

默认 DML 统计异步更新；统计值相同不代表数据相同。

并发写入下：执行前检查 3.8–4.6% 的使用在违规数据上作答，绑定快照一次也没有。

处理失效：三道关才发布修复，否则撤下

1 恢复结构

过滤后每个业务键一行，
且不丢任何业务键

2 候选唯一

没有候选或多个候选，
都撤下口径

3 通过回归

重新通过准入门槛，
用学习题比对数值

状态流水：修复成功

“状态 = 完成”是唯一能恢复一键一行的过滤；回归测试通过后发布新修订。

备份副本：撤下口径

“来源 = 主库”和“来源 = 备份”都能恢复粒度，结构上分不出真伪，所以撤下。

宁可撤下口径，也不发布可能错的修复；修复进行中的请求等待结果，不重复算。

MAVRA：智能体与 PostgreSQL 之间的中间层



对智能体来说只是多了两个工具： `find_metric` 返回当前有效的口径， `run_sql` 在使用时核对。

两层实验：先固定口径验证机制，再让真实智能体跑完整任务

1 机制层：口径固定，只换维护方式

配对回放：15 个学到的口径库（97 个口径）+ TPC-DS 模板库，各 11 种数据变化

并发写入：执行前检查 vs 绑定快照，确定性交错 + 随机并发

维护代价：定义级 / 通用缓存 / MAVRA，100 万–1600 万行，8–32 个脚本智能体

2 端到端：真实智能体完整任务

DeepSeek V4.1 Flash；5 个指标先学再用，只给指标名

11 种数据变化：状态流水、版本化更正、维表拉链、重复装载、日期键改写（条件能发现）；备份副本（修复有歧义）；4 种正常追加；单位换算（对照）

9 种方法 × 3 次独立运行，2586 道计分题；另有 216 个固定库会话量时间与 token

机制层回答“维护对不对、贵不贵”，端到端回答“对使用者有没有用”。

数据变化后：维护后的口径零错误答案，只看结构 645 个

维护方式	答对	答错	其中：条件能发现的变化下答错
只看结构	784	746	645
表级测试（dbt 式）	631	101	0
定义级重验	973	101	0
MAVRA	973	101	0

每种方法 1,530 道配对题； 101 个错误全部来自单位换算这个对照。

0 vs 645

条件能发现的变化下的错误答案
MAVRA vs 只看结构（学到的库）

0 vs 437

同样的对比，真实 TPC-DS 数据
（99 个官方模板导出的口径库）

表级测试也能隔离破坏，但不会修： MAVRA 多答对 342 题，来自修复或口径自带的过滤。

并发写入：执行前检查 4.6% 违规作答，绑定快照一次没有

3.8–4.6%

执行前检查：未通知的写入下
在违规数据上作答的使用

0 / 11,297

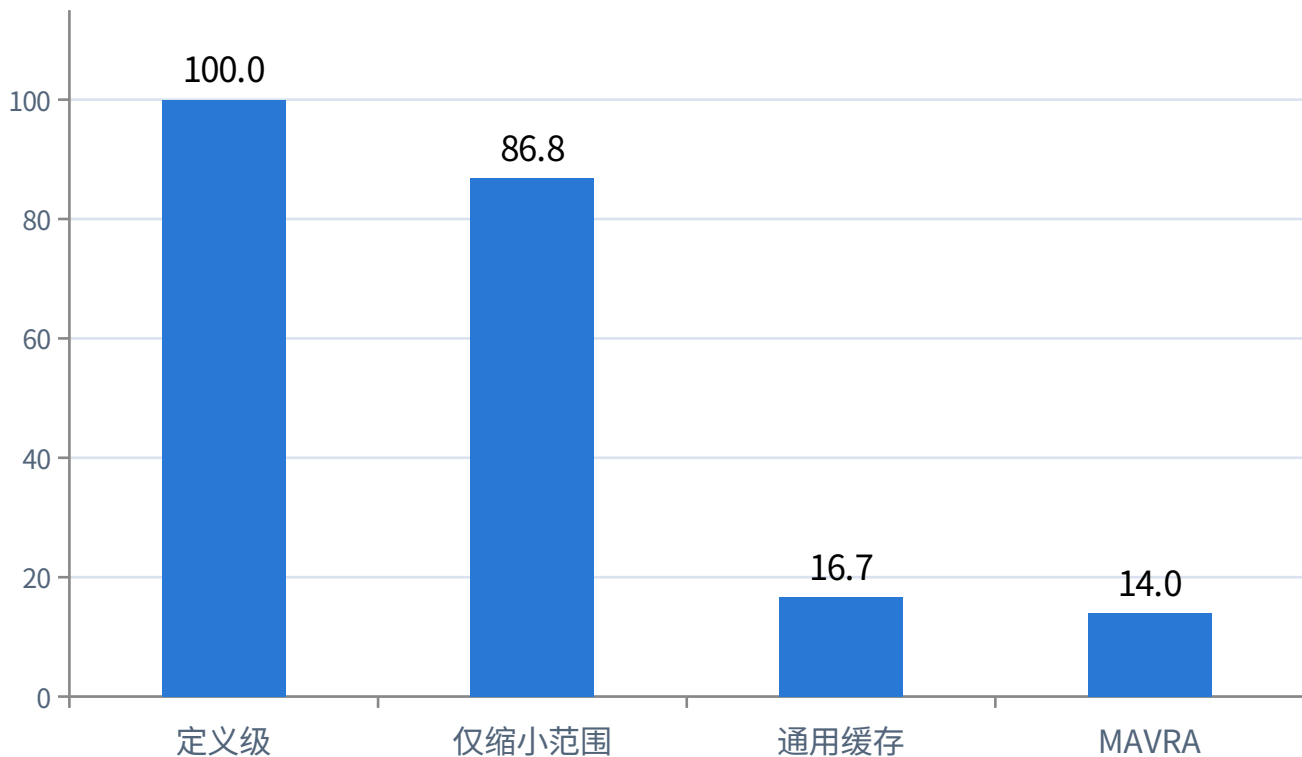
绑定快照：随机并发合计
在违规数据上作答的使用

代价	观测结果
读者中位延迟	26–29 ms ， 两种模式相同
单写者吞吐（相对无触发器）	0.70×
32 个并发写者：分片计数 / 不分片	0.97× / 0.36×

保证来自事务性版本：写入和版本递增在同一事务里提交，读者看到其一就看到其二。

维护代价：共享条件后，只需定义级重验的 14%

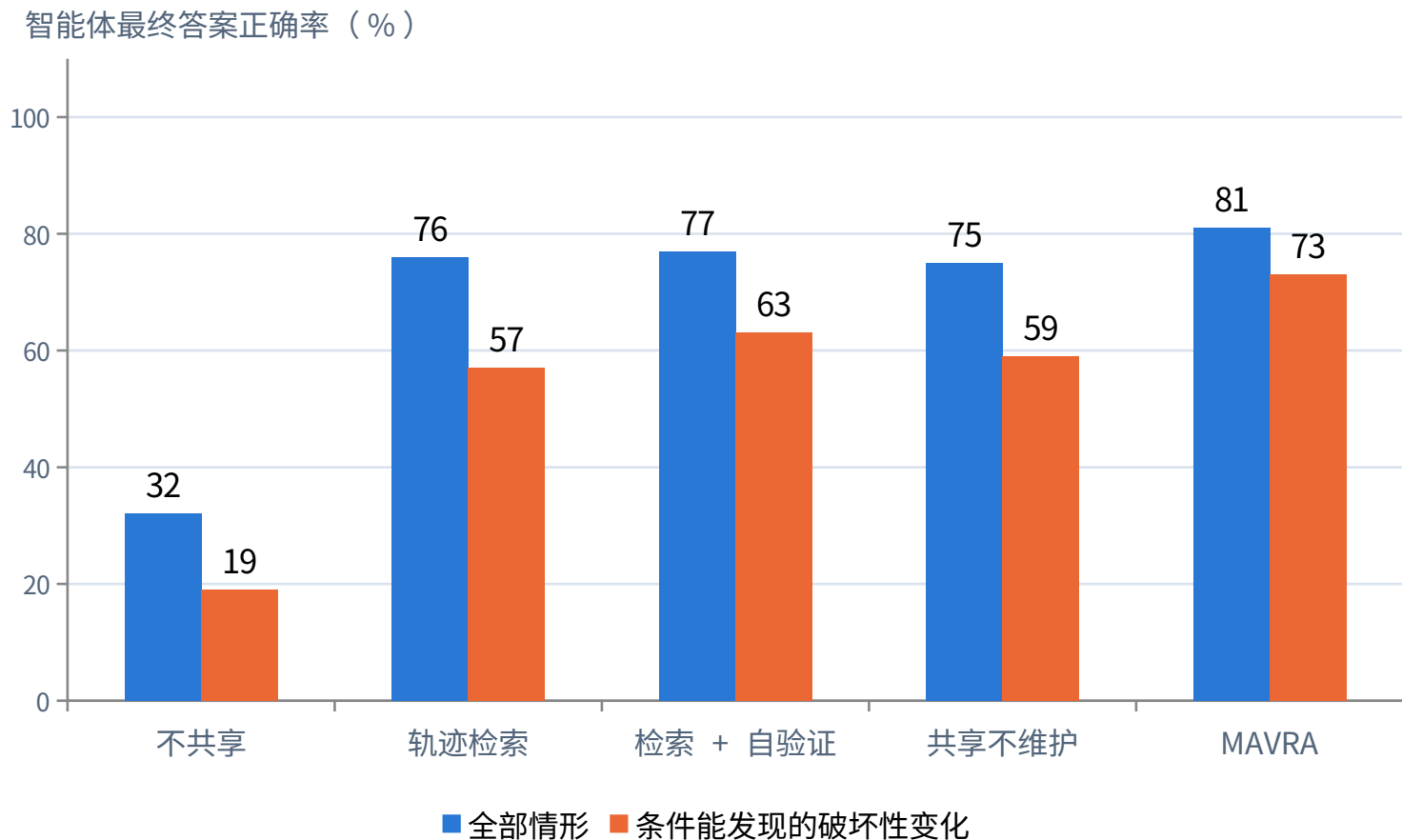
维护数据库时间，相对定义级 (%)



- 定义级随口径数增长：63.6 → 227.0 秒。
- 19 个口径：MAVRA 为定义级的 14.0%，通用缓存 16.7%。
- 100 万到 1600 万行：比例稳定在 12.0–13.0%。
- 口径之间没有共享条件时，所有方法都是定义级的 95–102%。

节省来自结论共享，通用版本缓存也能拿到；论文的贡献在条件、使用保证和修复规则。

端到端：共享把正确率从 32% 提到 75%，维护再拉开 16 个点



- 共享本身：32% → 75%；再加维护 81%。
- 破坏性变化：MAVRA 73% vs 轨迹检索 57%；预登记的差 +16（95% 区间 14-18）。
- 让检索的智能体自己检查：63%，仍落后 11 个点。

反例：备份副本下 MAVRA 撤下口径，这一组 61% 低于检索的 72%。

共享提高准确率；维护让共享的口径在数据变了之后仍然可靠。

对使用者：任务更快、调用更少、token 更少

方法	答对 / 54	平均完成 (秒)	LLM 调用 / 题	输入 token / 次尝试
不共享口径	16/54	64.47	5.43	16,329
定义级重验	54/54	32.97	3.76	9,227
通用版本缓存	54/54	31.97	3.74	9,137
MAVRA	54/54	37.68	3.83	9,689

-40.7%

MAVRA 相对不共享
每次尝试的输入 token

-27.7%

业务定义相同时
每题的结构查找次数

-72.7%

业务定义相同时
每题的 SQL 探查次数

节省来自共享：定义级
32.97 秒、通用缓存 31.97
秒，维护方式之间没有延迟
优势。

216 个全新会话：不共享 16/54 答对、64.47 秒； MAVRA 54/54 、 37.68 秒。

保证的范围，和现在还做不到的

1 保证只覆盖声明的使用

声明口径 + 绑定快照：条件在所读数据上成立。

未声明的 SQL 不在契约内；此前样本中 25.7% 的答案未声明口径，声明正确修订后仍有 1.3% 因 SQL 偏离而答错。

2 条件看不到取值语义

单位换算不违反任何条件；备份副本里两份总体都合法。

这两类要靠业务证据；修复也只会加过滤，还不会去重或重映射键。

3 评估的局限

数据变化由我们注入，不是真实更新历史。

端到端主实验一个模型、每种方法 3 次；更早一轮三个模型趋势一致。

下一步：取值层面的条件、识别未声明的使用、去重和重映射修复、真实更新历史。

论文贡献与当前进展

四项贡献

- ① 把共享口径建模为依赖四类可执行条件的可维护知识，并证明条件的充分性（命题 1）
- ② 把每次声明的使用绑定到查询自身快照上成立的证据，复用规则可靠（引理 1）
- ③ 有界修复算法：唯一、不丢键、过回归，否则撤下（算法 1）
- ④ 实现与两层评估：配对回放、TPC-DS、并发、代价、端到端

进展与下一步

- ✓ 论文完整初稿，投稿目标 SIGMOD 2027 研究 track
- ✓ 摘要、引言按“问题—做法—效果”重写，引用逐条核对
- ✓ 代码、工具和实验数据在仓库主线上，可复算
- 正文约 12.6 页，需压缩 0.6 页
- 10 月 7 日配额重置后补跑其他模型
- 真实更新历史、取值条件、未声明使用

方法、证明、系统和实验已经闭环；剩下的是压缩篇幅和补强评估。

让共享知识在变化的数据上继续成立

- ① 记录：从口径推出粒度、连接、时间角色、覆盖四类条件
- ② 维护：结论按数据版本保存，更新后只重查受影响的条件
- ③ 约束使用：查询只在条件成立的快照上执行
- ④ 处理失效：唯一且过回归才修复，否则撤下

想请老师把关的三个问题

1. 定位：聚焦“共享指标口径的有效性维护”，而不是更宽的“智能体共享记忆层”，这样切是否站得住？
2. 评估：端到端一个模型 × 3 次、数据变化由我们注入；补哪一项最有说服力？
3. 投稿：按 SIGMOD 2027 研究 track，正文再压 0.6 页；目标与时间是否合适？

形式化：定义、命题 1、引理 1 与算法 1

修订 $m^r = (B, I, C, E, r)$ 业务含义 · 结构化实现 · 条件集 · 证据 · 修订号；规范 SQL 由 I 编译

命题 1（充分性）

$B1$ 、 $B2$ 与 $C(m^r)$ 在快照 D_s 上成立 $\Rightarrow$ 对每个期间 p ，规范 SQL 返回角色日期落在 p 内的事件度量的聚合，每个事件恰好贡献一次；遗漏仅为日期键匹配不到日期维度的事件，其比例受覆盖条件约束。

引理 1（结论复用）

两个快照 s_1 、 s_2 中，条件 c 读到的每张表 T 的事务性版本相同 $\Rightarrow c(D_{\{s_1\}}) = c(D_{\{s_2\}})$ 。证明用快照嵌套与“写入和版本递增在同一事务提交”。

算法 1（有界修复）

粒度条件失效 $\rightarrow$ 在低基数列上枚举等值过滤 $\rightarrow$ 保留恢复一键一行且不丢业务键的候选 $\rightarrow$ 候选恰好一个 $\rightarrow$ 通过准入门槛 $G3$ – $G5$ 与回归 $G8$ $\rightarrow$ 发布新修订；任一步失败则撤下口径。

命题划定检测边界：取值变化、结构等价的总体、合法但错误的日期键不由结构条件证明。

修复参照决定自动化范围，唯一性规则挡住歧义

同一批配对题	判题学习查询作参照	智能体学习 SQL 作参照
1,470 道题：答对	933	780
新增错误	—	0（差别全部转为不可用）
端到端：破坏性变化下正确率	73%	74%

反例：追加备份副本

不要求候选唯一时可发布错误修复；要求唯一时两种参照均撤下口径。

修复期间等待

同时到达的不可用从 63 降为 0；后到的请求共享已有的维护结果。

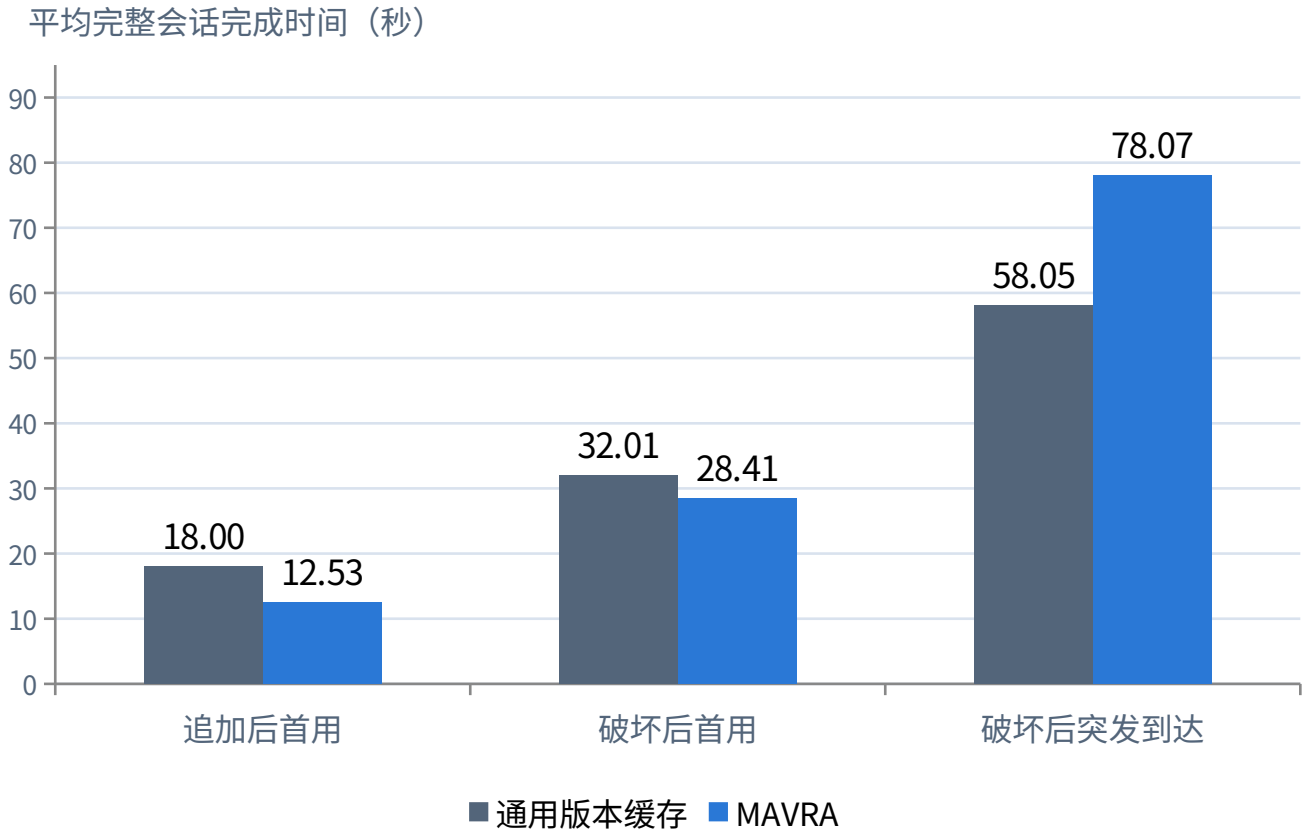
回归参照只有在变化触及学习期间、且参照反映区分列时，才提供额外分辨力。

各类方法分别覆盖生命周期的哪些环节

方法	正常更新	条件破坏后	使用时保证	维护单位
轨迹检索	复用 SQL	靠智能体自己发现	不绑定条件	检索到的成功轨迹
只看结构	保留	结构未变时漏检	结构 / 修订核对	结构指纹
表级测试（dbt 式）	保留	隔离相关表上的口径	执行前的测试	按表声明的测试
写入即撤销	撤下并重学	撤下并重学	执行前核对	每次写入
定义级 + 版本缓存	保留	同样的修复与回归	执行前检查	缓存的检查 SQL
MAVRA	保留	有界修复或撤下	可绑定同一快照	变化触及的不同条件

共享、验证和缓存是基础设施；本文研究的是口径的有效性生命周期。

完整会话延迟：更新后首用更快，突发到达仍有模型长尾



数据库工作量减少

MAVRA 比通用缓存少 25.3% 的数据库查询时间。

整体延迟优势不成立

全矩阵平均 37.68 对 31.97 秒；一次 356.23 秒的长尾中模型接口占 338.45 秒。

测量包括模型、工具与排队；数据库工作量和完整会话时间要分开报告。

应用重复的是查询，智能体重复的是知识

观察	应用负载： Redset	智能体： 100 个全新会话
复用对象	稳定的查询模板	表结构、粒度、连接等事实
重复程度	半数集群 80% 的查询完全重复	93% 的查找重复此前会话获得的事实
SQL 探查	34% 的读查询由结果缓存回答	38% 重复事实；逐字相同的 SQL 为 0
探索与验证	知识预先写在应用代码里	68% 的调用用于探索； 16% 的会话查键唯一性
错误方式	—	32 个错误答案全部来自成功执行的 SQL

共享能减少重复探索；支撑共享口径的事实仍需在更新后重验。